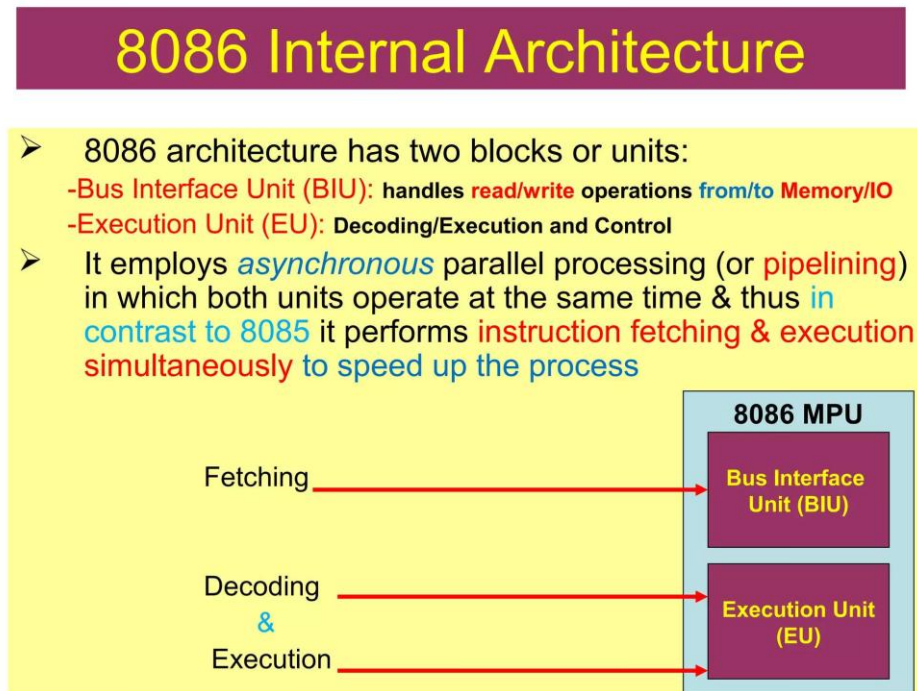


Question bank on Unit 4

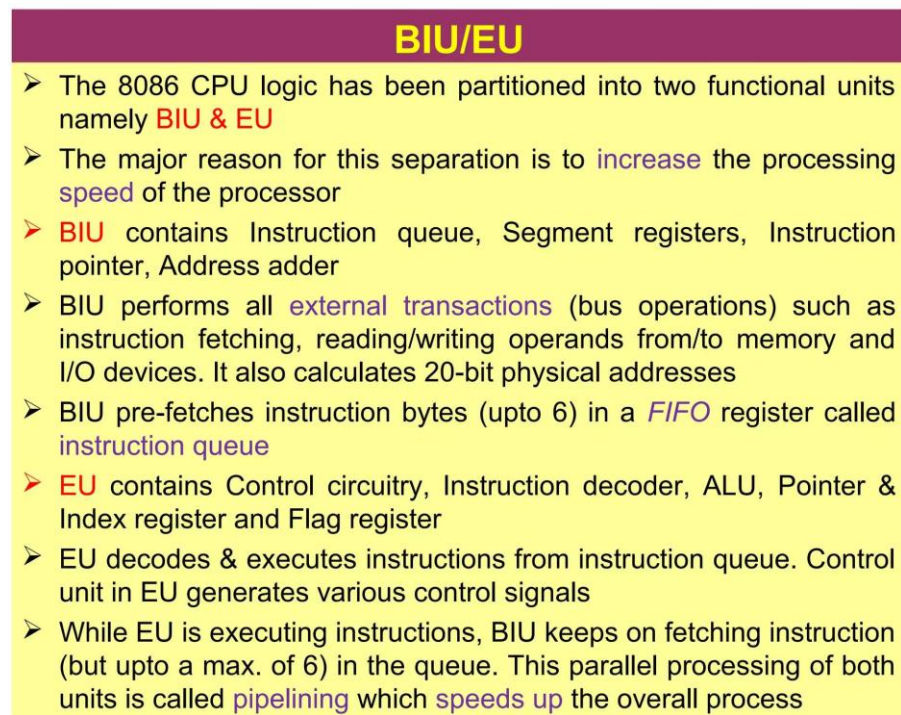
1. Explain the internal architecture of the 8086 microprocessor with the help of a neat block diagram.

Ans:



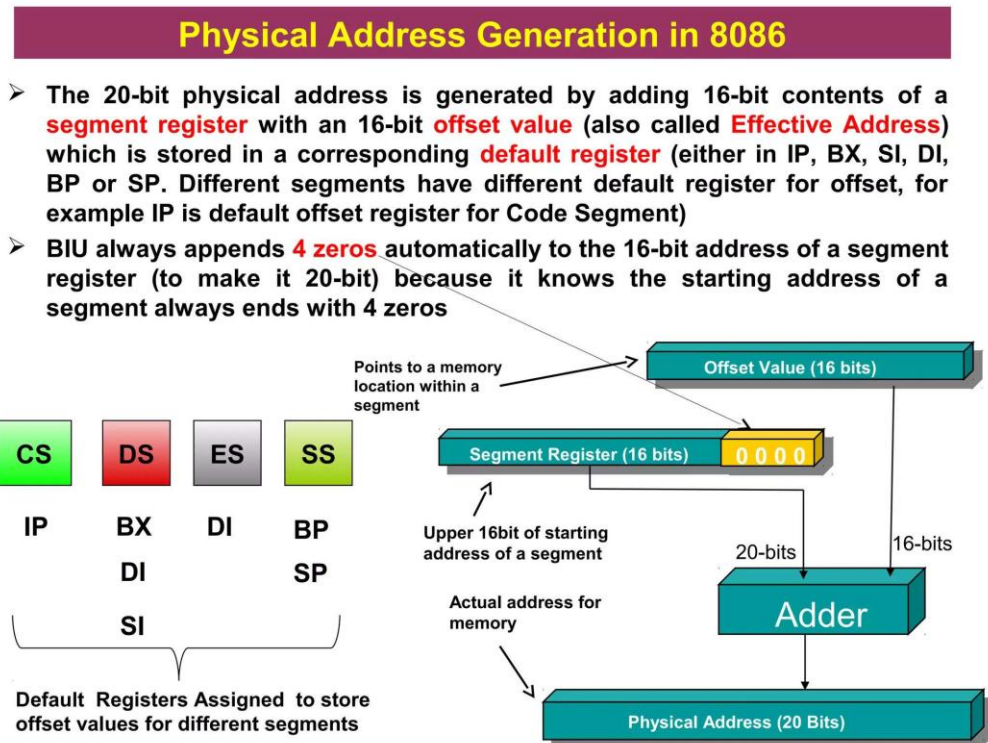
2. Explain the role of the EU and BIU in improving the performance of the 8086 processor.

Ans:



3. Define physical address in 8086. How is it generated from segment and offset addresses?

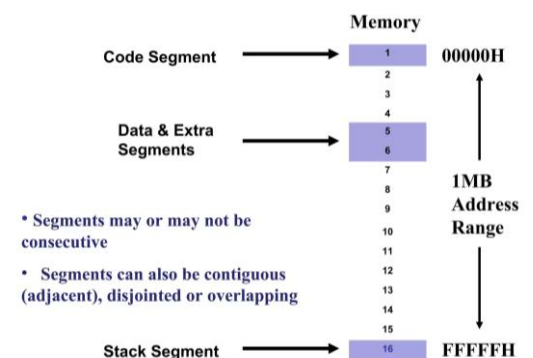
Ans:



4. Explain a segmentation scheme for a program requiring code, stack, and data segments, extra segments.

Ans:

- In 8086, a program's memory is organized into up to four active 64KB segments, which can be located anywhere in the 1MB memory space.
- Code Segment (CS): Stores the program's executable instructions. The IP register points to the offset of the next instruction here.
- Data Segment (DS): Stores the program's variables and data. The SI or DI registers often hold offsets within this segment.
- Stack Segment (SS): Used for the stack to store return addresses, parameters, and temporary data. The SP register points to the top of the stack.
- Extra Segment (ES): An additional data segment, primarily used as the destination for string manipulation instructions.
- This scheme allows for organization and protection, as code, data, and stack can be placed in different memory regions.



5. List and describe the different types of registers available in 8086.

Ans:

The 8086 has 14 registers, categorized as follows:

- General-Purpose Registers (16-bit, can be used as 8-bit):
 - AX (Accumulator): For arithmetic, I/O, multiplication, and division.
 - BX (Base): Can hold a memory offset in based addressing.
 - CX (Count): Used as a loop and shift/rotate counter.
 - DX (Data): Used for multiplication/division and I/O port addressing.
- Pointer and Index Registers (16-bit): Used to hold memory offsets.
 - SP (Stack Pointer): Points to the top of the stack.
 - BP (Base Pointer): Points to a location within the stack.
 - SI (Source Index): Used as a source pointer in string operations.
 - DI (Destination Index): Used as a destination pointer in string operations.
- Segment Registers (16-bit): Hold the base addresses of memory segments.
 - CS (Code Segment), DS (Data Segment), SS (Stack Segment), ES (Extra Segment).
- Instruction Pointer (IP): A 16-bit register holding the offset of the next instruction to execute in the code segment.
- Flag Register: A 16-bit register with 9 status and control flags.

6. Explain the role of segment registers and how they differ from general-purpose registers.

Ans:

- Role of Segment Registers: Their sole purpose is to hold the 16-bit base addresses of the 64KB memory segments (Code, Data, Stack, Extra). They are essential for the 8086's ability to address 1MB of memory.

- Differences:

Function: General-purpose registers (AX, BX, etc.) hold data, while segment registers hold segment base addresses.

Use: General-purpose registers are used in arithmetic and data manipulation. Segment registers are automatically used by the CPU to calculate the physical address for memory access (e.g., CS for instruction fetch, SS for stack operations).

Data Loading: You cannot load an immediate value directly into a segment register; you must first load it into a general-purpose register and then move it.

7. Show how AX, BX, CX, and DX can be accessed in parts (8-bit and 16-bit). Give examples.

Ans:

The 16-bit registers AX, BX, CX, and DX can each be accessed as two independent 8-bit registers: a high byte (AH, BH, CH, DH) and a low byte (AL, BL, CL, DL).

16-bit Example:

MOV AX, 4142H ; Loads the word 'AB' into AX. Now AH = 41H, AL = 42H.

8-bit Example:

MOV CX, 0000H ; Clear CX.

MOV CH, 0FFH ; Load FFH into the high byte. CX is now 0FF00H.

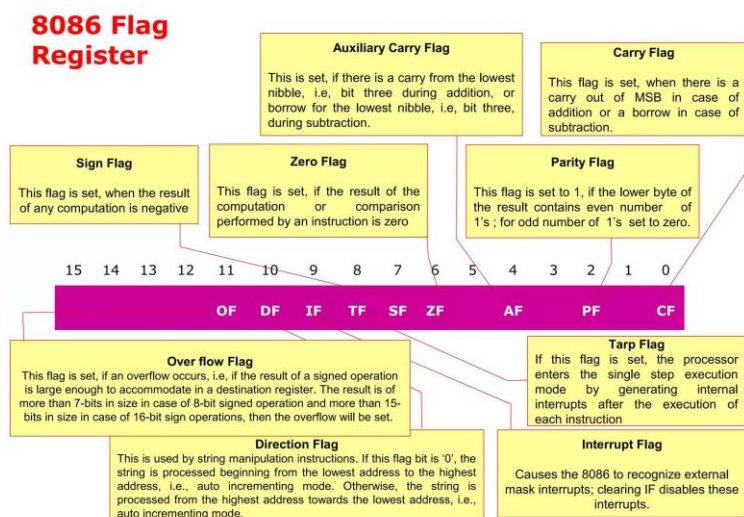
MOV CL, 1AH ; Load 1AH into the low byte. CX is now 0FF1AH.

8. Explain 8086 flag register in detail.

Ans:

- The 16-bit flag register has 9 active flags divided into two types:
- Conditional Flags (6): Reflect the result of the last arithmetic/logic operation.
- CF (Carry Flag): Set on carry/borrow from the MSB.
- PF (Parity Flag): Set if the result has an even number of 1s.
- AF (Auxiliary Carry Flag): For BCD arithmetic; set on carry from bit 3 to 4.
- ZF (Zero Flag): Set if the result is zero.
- SF (Sign Flag): Set if the result is negative (MSB is 1).
- OF (Overflow Flag): Set on a signed arithmetic overflow.

- Control Flags (3): Control the CPU's operation.
- TF (Trap Flag): Enables single-step mode for debugging.
- IF (Interrupt Flag): Enables or disables maskable interrupts.
- DF (Direction Flag): Controls the direction (increment/decrement) of SI and DI in string operations.



9. Explain Immediate, Direct, and Register addressing modes with suitable examples.

Ans:

The instruction will specify the name of the register which holds the data to be operated by the instruction.

Example:

MOV CL, DH

The content of 8-bit register DH is moved to another 8-bit register CL

$(CL) \leftarrow (DH)$

In immediate addressing mode, an 8-bit or 16-bit data is specified as part of the instruction

Example:

MOV DL, 08H

The 8-bit data (08_H) given in the instruction is moved to DL

$(DL) \leftarrow 08_H$

MOV AX, 0A9FH

The 16-bit data (0A9F_H) given in the instruction is moved to AX register

$(AX) \leftarrow 0A9F_H$

Here, the effective address of the memory location at which the data operand is stored is given in the instruction.

The effective address is just a 16-bit number written directly in the instruction.

Example:

**MOV BX, [1354H]
MOV BL, [0400H]**

The square brackets around the 1354_H denotes the contents of the memory location. When executed, this instruction will copy the contents of the memory location into BX register.

This addressing mode is called direct because the displacement of the operand from the segment base is specified directly in the instruction.

10. Explain Register Indirect, Indexed, and Relative addressing modes with suitable examples.

Ans:

In Register indirect addressing, name of the register which holds the effective address (EA) will be specified in the instruction.

Registers used to hold EA are any of the following registers:

BX, BP, DI and SI.

Content of the DS register is used for base address calculation.

Example:

MOV CX, [BX]

Note : Register/ memory enclosed in brackets refer to content of register/ memory

SI or DI register is used to hold an index value for memory data and a signed 8-bit or unsigned 16-bit displacement will be specified in the instruction.

Displacement is added to the index value in SI or DI register to obtain the EA.

In case of 8-bit displacement, it is sign extended to 16-bit before adding to the base value.

Example:

MOV CX, [SI + 0A2H]

1. Definition:

In relative addressing, the address of the instruction's operand (target) is found by adding a small offset (displacement) to the current program counter (PC).

2. How it works:

- The instruction gives an offset (positive or negative number).
- The processor adds this offset to the current PC value to get the actual address.

11. Explain base addressing, based indexed addressing, string addressing with suitable examples.

Ans:

In **Based Addressing**, BX or BP is used to hold the base value for **effective address** and a signed 8-bit or unsigned 16-bit displacement will be specified in the instruction.

When BX holds the base value of EA, 20-bit physical address is calculated from BX and DS.

When BP holds the base value of EA, BP and SS is used.

Example: Formula: Effective Address (EA) = Base Register + Displacement
Example: EA = BX + 08H

MOV AX, [BX + 08H]

MOV BX, 1000h ; BX holds base address
MOV AX, [BX+08h] ; AX gets value at base address + 8

In this addressing mode, the effective address of a program instruction is specified relative to Instruction Pointer (IP) by an 8-bit signed displacement.

In **Based Index Addressing**, the effective address is computed from the sum of a base register (BX or BP), an index register (SI or DI) and a displacement.

Example:

MOV DX, [BX + SI + 0AH]

000.

= +20.

- New address is employed in string operations to operate on string data.
- So, the effective address (EA) of source data is stored in SI register and the EA of destination is stored in DI register.

Segment register for calculating base address of source data is DS and that of the destination data is ES

Example: MOVSB BYTE

Operations:

Calculation of source memory location:

$EA = (SI)$ $BA = (DS) \times 16_{10}$ $MA = BA + EA$

Calculation of destination memory location:

$EA_E = (DI)$ $BA_E = (ES) \times 16_{10}$ $MA_E = BA_E + EA_E$

$(MAE) \leftarrow (MA)$

If $DF = 1$, then $(SI) \leftarrow (SI) - 1$ and $(DI) \leftarrow (DI) - 1$
If $DF = 0$, then $(SI) \leftarrow (SI) + 1$ and $(DI) \leftarrow (DI) + 1$

12. Explain direct and indirect I/O port addressing and implied addressing with suitable examples.

Ans:

These addressing modes are used to access data from standard I/O mapped devices or ports.

In **direct port addressing mode**, an 8-bit port address is directly specified in the instruction.

Example: `IN AL, [09H]`

Operations: $PORT_{addr} = 09_H$
 $(AL) \leftarrow (PORT)$

Content of port with address 09_H is moved to AL register

Instructions using this mode have no operands. The instruction itself will specify the data to be operated by the instruction.

Example: `CLC`

This clears the carry flag to zero.

1. **IN instruction:**
 - Used to read data from an input port into a CPU register.
 - General form:
 - `IN AL, port` → read a byte into AL from the given port.
 - `IN AX, port` → read a word into AX from the given port.
2. **Operand [09H]:**
 - Here 09H is the port address (in hexadecimal).
 - Square brackets are sometimes used in notation to show the port address (not always required, depends on assembler).
3. **Meaning:**
 - `IN AL, 09H` means:
Read one byte from input port with address 09H and store it in AL.

13. List different types of instructions supported by 8086 with examples.

Ans:

Data Transfer Instructions These instructions move data between registers, memory, and I/O ports without modifying it.

- **MOV (Move):** Copies data from a source to a destination.
 - Example: `MOV AX, BX` copies the content of register BX into register AX.
- **IN (Input):** Reads data from an I/O port into the accumulator.
 - Example: `IN AL, [09H]` reads a byte from port 09H into the AL register.

Arithmetic Instructions These instructions perform arithmetic operations like addition, subtraction, multiplication, and division.

- **ADD (Add):** Adds two operands.
 - (Example not in slides, but this is a standard arithmetic instruction).
- **MUL (Multiply):** Performs an unsigned multiplication.
 - (Example not in slides, but this is a standard arithmetic instruction).

Branch Instructions These instructions transfer the flow of execution to a different part of the program, either conditionally or unconditionally.

- **JZ (Jump if Zero):** Jumps to a target address if the Zero Flag (ZF) is set to 1.

- Example: JZ 0AH jumps 10 bytes forward if the result of a previous operation was zero.

Logic Instructions These instructions perform bitwise logical operations such as AND, OR, and XOR.

- **AND (Logical AND):** Performs a bitwise AND operation on two operands.
 - (Example not in slides, but this is a standard logic instruction).
- **OR (Logical OR):** Performs a bitwise OR operation.
 - (Example not in slides, but this is a standard logic instruction).

Bit Manipulation Instructions These instructions are used to set, clear, or test individual bits, including processor flags.

- **CLC (Clear Carry Flag):** Clears the Carry Flag (CF) to zero.
 - Example: CLC.

String Instructions These instructions are used to process sequences of data (strings) in memory.

- **MOVS/MOVSW (Move String):** Moves a byte or word from a source location (DS:SI) to a destination location (ES:DI).
 - Example: MOVS BYTE.

14. Explain the general format of 8086 instructions and timing.

Ans:

General Instruction Format

- The standard format for an 8086 assembly instruction is:
- Mnemonic Destination, Source
- Mnemonic: The operation code (e.g., MOV, ADD).
- Destination: The register or memory location where the result is stored.
- Source: The data for the operation, which can be a register, memory location, or a constant value.

Instruction Timing

- Instruction execution time is measured in bus cycles, with each cycle consisting of at least four clock ticks called T-states.

- Read Bus Cycle (Memory/IO Read)
- T1: The CPU outputs the address on the bus and asserts the Address Latch Enable (ALE) signal.
- T2: The Read (RD') signal goes low.
- T3: The CPU reads data from the bus. Slow devices can add wait states (TW) here.
- T4: The RD' signal goes high, ending the cycle.

Write Bus Cycle (Memory/IO Write)

- T1: The CPU outputs the address on the bus and asserts ALE.
- T2: The Write (WR') signal goes low, and the CPU places data on the bus.
- T3: Data is held on the bus to allow the device to capture it.
- T4: The WR' signal goes high, ending the cycle.

15. Explain data transfer instructions with examples.

Ans:

MOV (Move)

This is the most common data transfer instruction. It copies a byte or a word from a **source** operand to a **destination** operand.

- **Syntax:** MOV destination, source
- **Examples:**
 - **Register to Register:** MOV CL, DH copies the 8-bit content of the DH register into the CL register.
 - **Immediate to Register:** MOV AX, 0A9FH moves the 16-bit constant value 0A9FH directly into the AX register.
 - **Memory to Register:** MOV BX, [1354H] copies the 16-bit word from the memory location specified by the direct address 1354H into the BX register. The address is calculated using the Data Segment (DS) register by default.

IN / OUT (Input / Output)

These instructions are used to transfer data between the accumulator (AL or AX) and I/O ports.

- **Syntax:** IN accumulator, port_address
- **Function:** The IN instruction reads data from a specified input port and stores it in the accumulator.
- **Example:** IN AL, [09H] reads one byte from the input port with address 09H and stores it in the AL register.

The OUT instruction (not explicitly shown in the examples, but related) performs the reverse operation, sending data from the accumulator to an output port.

16. Explain arithmetic instructions with examples.

Ans:

Key Arithmetic Instructions (General Examples)

- **ADD (Add):** Adds the source operand to the destination operand and stores the result in the destination.
 - **Example:** ADD AX, BX would compute $AX = AX + BX$.
- **SUB (Subtract):** Subtracts the source operand from the destination operand.
 - **Example:** SUB CX, 10H would compute $CX = CX - 16$.
- **MUL (Multiply):** Performs an unsigned multiplication of the source operand with the accumulator (AL or AX).
 - **Example:** MUL BL would multiply the value in AL by the value in BL, storing the 16-bit result in AX.
- **DIV (Divide):** Performs an unsigned division. The dividend is in the accumulator (AX or DX:AX).
 - **Example:** DIV CL would divide the 16-bit value in AX by the 8-bit value in CL. The quotient is stored in AL, and the remainder is in AH.

17. Explain branch instructions with examples.

Ans:

JZ (Jump if Zero)

The JZ instruction is a **conditional jump** that transfers program control to a new address only if the **Zero Flag (ZF) is set to 1**. The Zero Flag is typically set when the result of a preceding arithmetic or logical operation is zero.

- **Function:** If $ZF = 1$, the program "jumps" to the new address. If $ZF = 0$, the instruction is ignored, and the program continues to the very next instruction in sequence.
- **Address Calculation:** The target address is specified relative to the current Instruction Pointer (IP). The instruction contains a signed 8-bit displacement, which is added to the IP to calculate the new address.
- **Example:**

JZ 0AH

In this example, if the Zero Flag is set (ZF=1), the processor will add 0AH (10 in decimal) to the Instruction Pointer, causing the program to jump 10 bytes forward and execute the instruction at that new location .

18. Explain logic instructions with examples.

Ans:

AND

The AND instruction performs a bit-by-bit logical AND operation between the source and destination operands and stores the result in the destination. It's often used to clear specific bits (masking).

- **Example:** AND AL, 0FH
 - If AL was 1101 1011, the operation would be 1101 1011 AND 0000 1111.
 - The result in AL would be 0000 1011.

OR

The OR instruction performs a bit-by-bit logical OR operation and stores the result in the destination. It is often used to set specific bits.

- **Example:** OR BL, 80H
 - This ensures the most significant bit of BL is set to 1, regardless of its previous state.

XOR

The XOR (Exclusive OR) instruction performs a bit-by-bit XOR operation. A bit in the result is 1 if the corresponding bits in the operands are different. It's useful for toggling specific bits or clearing a register by XORing it with itself.

- **Example:** XOR CX, CX
 - This is a common and efficient way to set the CX register to zero.

NOT

The NOT instruction inverts every bit of its single operand (a one's complement).

- **Example:** NOT DX
 - If DX contained 0000 1111 0000 1111, after the NOT operation, it would become 1111 0000 1111 0000.

19. Explain bit manipulation instructions with examples.

Ans:

Logical Operations

These instructions perform bitwise AND, OR, XOR, and NOT operations. They are often used to selectively clear (mask), set, or invert specific bits.

Example (Masking with AND): To isolate the lower 4 bits of the AL register, you can use AND.

AND AL, 0FH ; Clears the upper 4 bits of AL, leaving the lower 4 unchanged.

Shift Operations

Shift instructions move the bits of an operand to the left or right, which is useful for multiplication or division by powers of 2.

Example (SHL - Shift Left): The presentation gives an example of shifting a number left by one position. SHL AX, 1

If AX contains 0000 0000 0000 1101 (13), after this instruction, AX will contain 0000 0000 0001 1010 (26). This is equivalent to multiplying by 2.

Flag Manipulation Instructions

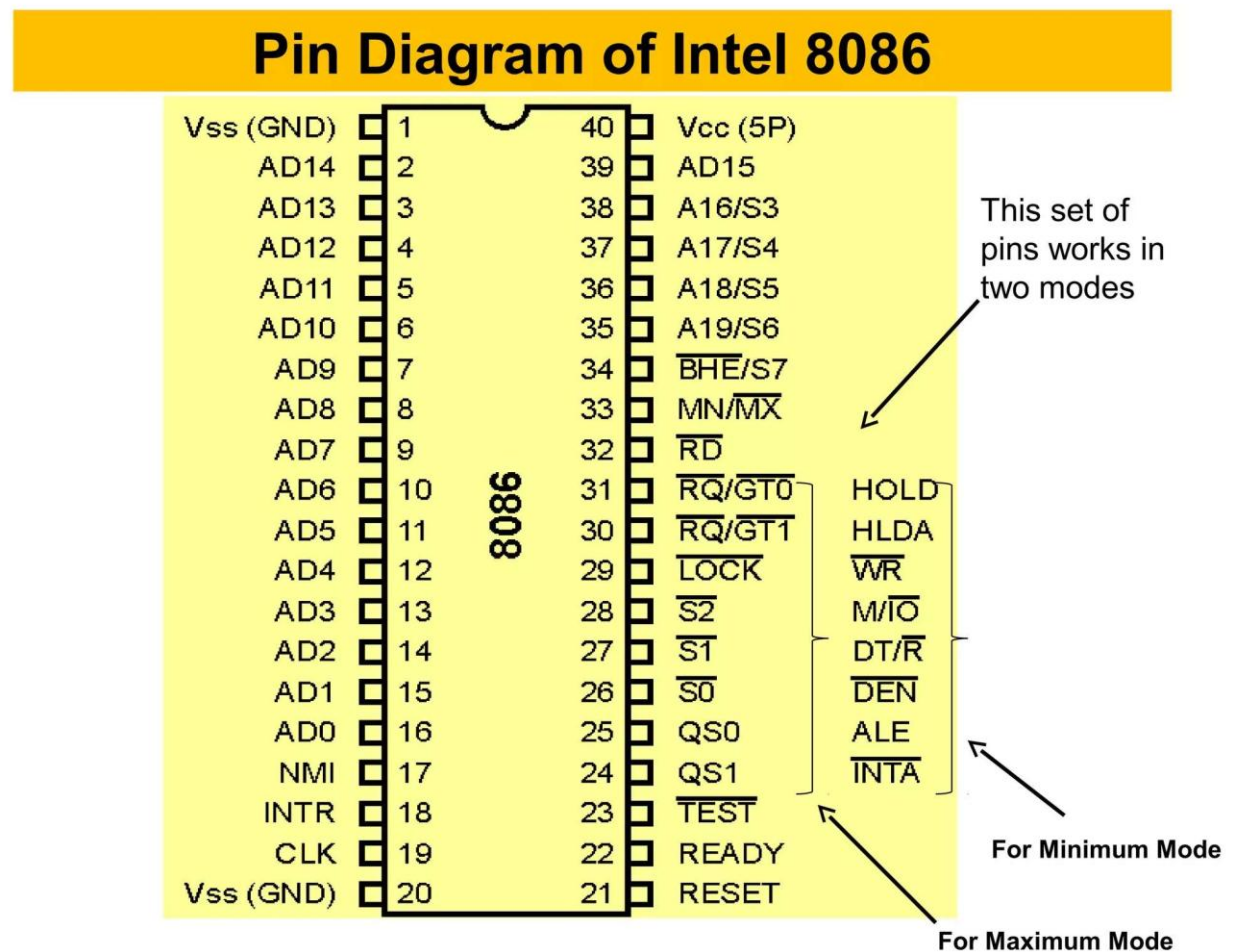
These instructions directly change the state of specific flags in the flag register.

Example (CLC - Clear Carry Flag): This instruction sets the Carry Flag (CF) to 0. It is often used before an addition operation to ensure there is no initial carry.

CLC ; Sets CF = 0

Example (STD - Set Direction Flag): This instruction sets the Direction Flag (DF) to 1. This causes string operations (like MOVSB) to process data from higher memory addresses to lower ones (auto-decrementing SI and DI).

20. Draw the pin diagram of 8086 and label the pins.



21. Explain the function of multiplexed address/data bus pins (AD0–AD15).

Ans:

- The AD0-AD15 pins are multiplexed to reduce the chip's pin count.
- In the first part of a bus cycle (T1): They carry the lower 16 bits of the memory/IO address. The ALE signal is used to latch this address externally.
- For the rest of the cycle: They function as the 16-bit data bus, transferring data to or from the processor.

22. Explain Minimum Mode and Maximum Mode operation of 8086.

Ans:

- **Minimum Mode:** Selected when the $\text{MN}/\overline{\text{MX}}$ pin is HIGH. This is for single-processor systems. The 8086 generates all bus control signals (like $\overline{\text{RD}}$, $\overline{\text{WR}}$, ALE) itself. It's a simpler, less expensive configuration.

- Maximum Mode: Selected when MN/MX' is LOW. This is for multi-processor systems (e.g., with an 8087 coprocessor). The 8086 outputs status codes (S0-S2) which are decoded by an external 8288 bus controller to generate the necessary control signals. This mode provides signals for bus arbitration and is more complex and costly.

23. Explain the following pins of 8086: AD0-AD15, BHE'/S7,Ready,RD'

Ans:

- AD0-AD15: Multiplexed address and data lines.
- BHE' / S7: Bus High Enable. An active-low signal used to enable the upper byte of the data bus (D8-D15) for 16-bit or odd-address byte transfers.
- READY: An input from slow memory/IO devices. If low, it forces the CPU into a wait state until the device is ready to transfer data.
- RD' (Read): An active-low output signal asserted by the CPU to read data from memory or an I/O port.

24. Explain the following pins of 8086: RESET,INTR,NMI,TEST

Ans:

- RESET: An active-high input that terminates current activity and forces the CPU to start execution from address FFFF0H.
- INTR (Interrupt Request): A maskable interrupt input. External devices use it to request service from the CPU.
- NMI (Non-Maskable Interrupt): An edge-triggered interrupt that cannot be disabled. Used for critical events.
- TEST': An input checked by the WAIT instruction, used for synchronization with a coprocessor like the 8087.

25. Explain the following pins of 8086:CLK,VCC,INTA',ALE

Ans:

- CLK (Clock): Provides the basic timing signal for all processor operations.
- VCC: The +5V power supply pin.
- INTA' (Interrupt Acknowledge): An active-low output signal in Minimum Mode that signals to a device that its interrupt request has been accepted.
- ALE (Address Latch Enable): An output signal indicating that a valid address is on the multiplexed bus, used to trigger an external latch.

26. Explain the following pins of 8086:LOCK', RQ'/GT',HOLD,HLDA

Ans:

- LOCK': An active-low output in Maximum Mode that prevents other processors from taking the bus, ensuring an instruction executes atomically.
- RQ'/GT' (Request/Grant): Bidirectional pins in Maximum Mode for bus arbitration between multiple processors.
- HOLD: An input in Minimum Mode used by devices like DMA controllers to request control of the system bus.
- HLDA (Hold Acknowledge): An output in Minimum Mode that signals that the CPU has granted the bus to the requesting device.

27. Explain the following pins of 8086: WR', MN/MX', DEN', DT/R'

Ans:

- WR' (Write): An active-low output in Minimum Mode that signals a write operation to memory or I/O.
- MN/MX': The input pin that selects between Minimum (high) and Maximum (low) operating modes.
- DEN' (Data Enable): An active-low output in Minimum Mode used to enable the external data bus transceiver.
- DT/R' (Data Transmit/Receive): An output in Minimum Mode that controls the direction of data flow through the transceiver. High for transmit (write), low for receive (read).